

G04 / PROTECTED ANSWERS

G04 REVIEW A

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

- Read this first 3
- Pipeline Engineering boundary after M10 3
- Dataset: water_meter_batches.csv 3
- 1. Architecture and boundaries 3
- 2. Container design 3
- 3. Partitioned orchestration 3
- 4. Reconciliation 3
- 5. Failure/recovery 3
- 6. Operational defense 3
- Gate decision 3
- Reproduce the numeric controls 4
- One concrete runtime design 4
- Scheduling and recovery trace 5

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

Pipeline Engineering boundary after M10

Protected review - open only after the corresponding Gate attempt is saved.

Dataset: water_meter_batches.csv

1. Architecture and boundaries

Strong answer: source evidence preserved; data stages have explicit contracts; Docker packages runtime but not mutable business state; Airflow coordinates partitioned tasks without owning raw transformation semantics.

2. Container design

Strong answer: image vs runtime config separated; service name used inside Compose network; volume/bind mounts justified; no real credential literal; down -v identified as destructive reset.

3. Partitioned orchestration

Expected dates Sep 1-3. Historical run uses historical date. Retry only transient/idempotent work. Small metadata may use task returns; bulk records stay in files/tables.

4. Reconciliation

Controls: Sep1 rows=3 total=300; Sep2 rows=3 total=320; Sep3 rows=4 total=415; overall rows=10 total=1035. IDs may repeat across dates because grain is meter-event/date, but duplicate same meter/date must be deliberate/validated.

5. Failure/recovery

Transient retry preserves attempt evidence; deterministic bad data fails/repairs rather than loops. Reprocessing Sep2 targets only Sep2 and converges/upserts/replaces its partition without multiplying totals.

6. Operational defense

Strong defense distinguishes static/simulator evidence from real Docker/Airflow runtime and names the pending learner-PC proofs honestly.

Gate decision

Use this only to diagnose the failed competency. Close the review before the changed repair/retest. Preserve the original attempt.

Reproduce the numeric controls

This complete function validates the full fixture before returning a report. Invalid input raises before publication. A declares one meter reading per meter/date; B declares one dispatch per dispatch_id, even across dates. A repeated meter on a different date is legitimate. Run from the gate folder using the corresponding copied file.

```
import csv
import json
from collections import defaultdict
from datetime import date
from decimal import Decimal
from pathlib import Path

def controls(source, variant):
    id_col, value_col = (
        ("meter_id", "liters") if variant == "A" else ("dispatch_id", "minutes")
    )
    seen, groups = set(), defaultdict(lambda: {"rows": 0, "total": Decimal(0)})
    with Path(source).open(newline="") as f:
        rows = list(csv.DictReader(f))
    for row in rows:
        day = date.fromisoformat(row["event_date"]).isoformat()
        key = (row[id_col], day) if variant == "A" else row[id_col]
        value = Decimal(row[value_col])
        if not row[id_col] or key in seen or not value.is_finite() or value <= 0:
            raise ValueError(f"Invalid or duplicate source record: {row}")
        seen.add(key)
        groups[day]["rows"] += 1
        groups[day]["total"] += value
    total = sum((x["total"] for x in groups.values()), Decimal(0))
    return {
        "rows": len(rows),
        "total": str(total),
        "days": {
            day: {"rows": x["rows"], "total": str(x["total"])}
            for day, x in sorted(groups.items())
        },
    }
```

For A call controls("practice/assessment/practical/water_meter_batches.csv", "A"). For B call controls("practice/assessment/retest/clinic_dispatch.csv", "B"). Save the JSON report with json.dumps(result, indent=2). The reported totals must match the preceding worked controls. Adding a duplicate copied row must raise instead of returning a publishable report.

One concrete runtime design

Use a Python worker image with /app as WORKDIR. Copy the validator code into the image. Mount source files read-only at /data and persistent reports at /output. The worker is a command-line program, so it needs no host port. If a database service is added, connect to its Compose service name such as db:5432 inside that network; localhost would refer to the worker container itself. Keep the Airflow deployment and its metadata database in the M10 setup route rather than pretending one Python container is Airflow.

```
FROM python:3.12-slim
WORKDIR /app
COPY worker.py /app/worker.py
ENTRYPOINT ["python", "/app/worker.py"]
```

The worker.py entry point reads runtime arguments INPUT, DATE and OUTPUT, validates the source, selects the requested partition, writes a temporary output and atomically replaces only that partition file after its controls pass. Its business code belongs in source control; the generated report does not belong in the image. For an external database, use a transaction or an idempotent upsert keyed by the partition/event identity rather than claiming a local rename makes a remote system transactional.

Scheduling and recovery trace

Use the Airflow 3 setup and DAG pattern from M10. A daily task gets the requested date from `data_interval_start`, not `date.today()`. For A, the run covering [2026-09-02 00:00 UTC, 2026-09-03 00:00 UTC) selects Sep2's three rows and total 320. Validate -> publish are separate tasks; pass the small report path and control summary between them, not bulk source rows. Reprocessing that interval replaces/upserts Sep2 only; Sep1 and Sep3 controls stay 300 and 415.

A temporary source-unavailable error can receive a bounded retry because acquisition and publication have stable identities. A duplicate business key or bad number is deterministic: stop, retain the source/rejection evidence and repair the input or contract. Waiting on an upstream file can use a bounded sensor; a timeout must be visible, not an infinite loop.

In B, a timed-out external publish has unknown effect. Check the publish request's stable idempotency key or query the destination's durable status. If already committed, reconcile and record completion; if absent, retry with the same identity; if status cannot be established, stop for investigation. Never create a new request identity just to make the retry succeed. This is a worked design, not a claim that Docker or Airflow was executed during authoring.